

Contrastive Learning

1. แนวคิดพื้นฐาน

- Contrastive Learning คือการเรียนรู้โดยการเปรียบเทียบความแตกต่าง
- จุดประสงค์หลักคือการเรียนรู้การแทนข้อมูล (representation) ที่มีความหมาย โดยไม่ต้องมีการติดฉลาก (label) จำนวนมาก
- ใช้หลักการที่ว่าข้อมูลที่คล้ายกันควรมี representation ที่ใกล้เคียงกัน และข้อมูลที่ต่างกันควรมี representation ที่แตกต่างกัน

2. องค์ประกอบสำคัญ

- Positive Pairs: คู่ข้อมูลที่ควรมีความคล้ายกัน เช่น ภาพเดียวกันที่ถูกแปลงด้วยวิธีต่างๆ
- Negative Pairs: คู่ข้อมูลที่ควรมีความแตกต่างกัน
- Encoder: โมเดลที่ใช้แปลงข้อมูลเป็น representation
- Loss Function: ฟังก์ชันที่ใช้วัดความแตกต่างระหว่าง representation

3. วิธีการทำงาน

1. สร้าง positive pairs จากข้อมูลต้นฉบับ
2. ใช้ data augmentation สร้างรูปแบบที่แตกต่างของข้อมูลเดียวกัน
3. ส่งข้อมูลผ่าน encoder เพื่อได้ representation
4. คำนวณ loss โดยพยายามทำให้:
 - representation ของ positive pairs ใกล้เคียงกัน
 - representation ของ negative pairs แตกต่างกัน

5. ข้อดี

- ไม่ต้องการข้อมูลที่มี label จำนวนมาก
- สามารถเรียนรู้ feature ที่มีประโยชน์ได้โดยอัตโนมัติ
- ช่วยเพิ่มประสิทธิภาพในงาน downstream tasks

- ทนทานต่อ noise ในข้อมูล

6. ข้อควรระวัง

- การเลือก augmentation ที่เหมาะสมมีความสำคัญมาก
- ต้องการ batch size ที่ใหญ่พอสมควร
- การเลือก negative samples ที่ดีมีผลต่อประสิทธิภาพ
- อาจต้องการทรัพยากรการคำนวณมาก

Loss Functions สำหรับ Contrastive Learning

ในการเรียนรู้ representation ที่มีประสิทธิภาพ เรามักต้องการให้ตัวอย่างที่มี label เดียวกัน (positive pairs) มี embedding ที่ใกล้กัน ในขณะที่ตัวอย่างที่มี label ต่างกัน (negative pairs) มี embedding ที่ห่างออกไป เพื่อให้การจำแนกหรือ clustering ทำได้ดี หลาย loss functions ถูกออกแบบมาเพื่อตอบโจทย์นี้ โดยในที่นี้เราจะอธิบายแนวคิดและตัวอย่างการใช้งานของ loss แต่ละแบบ ดังนี้

1. Contrastive Loss หรือ Max Margin Contrastive Loss

Contrastive Loss เป็นพื้นฐานของการทำ contrastive learning โดยมีหลักการคือ

- ถ้าคู่ข้อมูลมี label เดียวกัน ($y = 1$) จะลดระยะห่างของ embeddings โดยใช้ loss เป็นค่า $(D)^2$
- ถ้าคู่ข้อมูลมี label ต่างกัน ($y = 0$) จะผลักดันให้ระยะห่างมากกว่า margin โดยใช้ loss เป็น $(\max(0, \text{margin} - D))^2$

โดยที่ D คือระยะห่าง (เช่น Euclidean distance) ระหว่าง embedding ของตัวอย่างทั้งสอง

สูตรสรุปคือ:

$$\text{loss} = y \cdot (D)^2 + (1 - y) \cdot (\max(0, \text{margin} - D))^2$$

ตัวอย่างโค้ดด้วย PyTorch:

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class ContrastiveLoss(nn.Module):
6     def __init__(self, margin=1.0):
7         super(ContrastiveLoss, self).__init__()
8         self.margin = margin
9
10    def forward(self, output1, output2, label):
11        # output1, output2: embeddings ขนาด [batch_size,
12        embedding_dim]
```

```

12         # label: tensor ที่มีค่า 1 ถ้าเป็นคู่ที่เหมือนกัน, 0 ถ้าไม่เหมือนกัน
13         euclidean_distance = F.pairwise_distance(output1, output2)
14         loss_similar = label * torch.pow(euclidean_distance, 2)
15         loss_dissimilar = (1 - label) *
            torch.pow(torch.clamp(self.margin - euclidean_distance, min=0.0), 2)
16         loss = torch.mean(loss_similar + loss_dissimilar)
17         return loss

```

ตัวอย่างการใช้งาน

```

1  output1 = torch.randn(4, 5)
2  output2 = torch.randn(4, 5)
3  labels = torch.tensor([1, 0, 1, 0], dtype=torch.float)

4  criterion = ContrastiveLoss(margin=1.0)
5  loss = criterion(output1, output2, labels)
6  print("Contrastive Loss:", loss.item())

```

2. Triplet Loss

Triplet Loss ขยายแนวคิดโดยใช้ชุดข้อมูล 3 ตัว คือ

- Anchor: ตัวอย่างหลัก
- Positive: ตัวอย่างที่มี label เดียวกับ Anchor
- Negative: ตัวอย่างที่มี label ต่างจาก Anchor

โดยมีวัตถุประสงค์ให้ระยะห่างระหว่าง (Anchor, Positive) น้อยกว่าระยะห่างระหว่าง (Anchor, Negative) อย่างน้อย margin หนึ่งหน่วย

สูตรพื้นฐานคือ:

$$loss = \max(0, d(anchor, positive) - d(anchor, negative) + margin)$$

โดยที่ $d(\cdot, \cdot)$ มักใช้ Euclidean distance

ตัวอย่างโค้ดด้วย PyTorch:

```

1  import torch

```

```

4  Computer vision, ไพโรสันต์ ผดุงเวียง

```

```

2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class TripletLoss(nn.Module):
6     def __init__(self, margin=1.0):
7         super(TripletLoss, self).__init__()
8         self.margin = margin
9
10    def forward(self, anchor, positive, negative):
11        # anchor, positive, negative: embeddings ขนาด [batch_size,
        embedding_dim]
12        distance_positive = F.pairwise_distance(anchor, positive)
13        distance_negative = F.pairwise_distance(anchor, negative)
14        losses = torch.relu(distance_positive - distance_negative +
        self.margin)
15        return losses.mean()
16

```

ตัวอย่างการใช้งาน Triplet Loss

```

1    anchor = torch.randn(4, 5)
2    positive = torch.randn(4, 5)
3    negative = torch.randn(4, 5)
4
5    triplet_criterion = TripletLoss(margin=1.0)
6    triplet_loss = triplet_criterion(anchor, positive, negative)
7    print("Triplet Loss:", triplet_loss.item())

```

3. Multi-class N-pair Loss

Multi-class N-pair Loss เป็นการขยายแนวคิดของ Triplet Loss โดยในแต่ละ mini-batch จะมีการเปรียบเทียบ anchor กับ positive พร้อมกับเปรียบเทียบกับ negative samples หลายตัวพร้อมกัน วิธีนี้ช่วยให้ข้อมูล negative samples ได้อย่างครอบคลุมมากขึ้น

หลักการคือ เมื่อเลือกคู่ (anchor, positive) ที่มี label เดียวกัน เราจะคำนวณ similarity matrix ระหว่าง anchor กับ positive โดยใช้ inner product (หรือ cosine similarity เมื่อ normalize embeddings) จากนั้น loss จะถูกคำนวณในรูปแบบ softmax-based โดยมีสูตรพื้นฐาน:

$$L = (1/N) \sum_i \log \left[1 + \sum_j \exp \left(f(x_i)^T f(x_j^-) - f(x_i)^T f(x_i^+) \right) \right]$$

โดยที่ $f(x_i)$ คือ embedding ของ anchor, $f(x_i^+)$ คือ embedding ของ positive sample และ $f(x_i^-)$ คือ embedding ของ negative sample

ตัวอย่างโค้ดด้วย PyTorch:

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class NPairLoss(nn.Module):
6     def __init__(self):
7         super(NPairLoss, self).__init__()
8
9     def forward(self, anchor, positive):
10         # anchor และ positive ควรมี shape [batch_size, embedding_dim]
11         batch_size = anchor.size(0)
12         # คำนวณ similarity matrix ระหว่าง anchor กับ positive
13         similarity_matrix = torch.matmul(anchor, positive.t())
14
15         # ค่า similarity ของคู่ที่จับคู่ (anchor กับ positive) อยู่ที่ diagonal
16         pos_sim = torch.diag(similarity_matrix).view(batch_size, 1)
17
18         # คำนวณ loss สำหรับแต่ละ anchor โดยเปรียบเทียบคู่ที่ถูกต้องกับคู่ที่ผิด
19         loss_matrix = similarity_matrix - pos_sim
20         loss = torch.mean(torch.log(1 +
21                               torch.sum(torch.exp(loss_matrix), dim=1)))
22         return loss
```

```
1 anchor = torch.randn(4, 5)
2 positive = torch.randn(4, 5)
3
4 npair_criterion = NPairLoss()
5 loss = npair_criterion(anchor, positive)
6 print("Multi-class N-pair Loss:", loss.item())
```

5. Supervised NT-Xent Loss

Supervised NT-Xent Loss (Normalized Temperature-scaled Cross Entropy Loss) เป็นการต่อยอดจาก NT-Xent loss แบบ self-supervised (ที่ใช้ใน SimCLR) โดยเพิ่มข้อมูล label เข้ามาช่วยในการกำหนดคู่ positive และ negative

ขั้นตอนหลักประกอบด้วย:

1. Normalization ของ Embeddings: ทำให้ embeddings มี norm เท่ากับ 1 เพื่อใช้ cosine similarity
2. คำนวณ Similarity Matrix: คำนวณ cosine similarity ระหว่างทุกคู่ใน batch และปรับด้วย temperature parameter (τ)
3. จัดกลุ่ม Positive และ Negative: ใช้ข้อมูล label เพื่อระบุว่า sample ไหนเป็น positive pair (label เดียวกัน) และ negative pair (label ต่างกัน)
4. คำนวณ Loss: สำหรับแต่ละ sample i , ใช้ cross entropy loss ที่พยายามให้ similarity ระหว่าง sample กับ positive สูงที่สุด โดยมีสูตรประมาณว่า:

$$L_i = -(1/|P(i)|) \sum_{p \in P(i)} \log \left[\exp(\text{sim}(z_i, z_p)) / \sum_a \exp(\text{sim}(z_i, z_a)) \right]$$

โดยที่ a วิ่งผ่านตัวอย่างทั้งหมดใน batch (ยกเว้นตัวเอง)

ตัวอย่างโค้ดด้วย PyTorch:

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class SupNTXentLoss(nn.Module):
6     def __init__(self, temperature=0.5):
7         super(SupNTXentLoss, self).__init__()
8         self.temperature = temperature
9
10    def forward(self, features, labels):
11        """
```

```

12         features: Tensor shape [batch_size, embedding_dim]
13         labels: Tensor shape [batch_size] (ชนิดข้อมูล int หรือ long)
14         """
15         device = features.device
16         batch_size = features.shape[0]
17
18         # Normalize features สำหรับ cosine similarity
19         features = F.normalize(features, p=2, dim=1)
20
21         # คำนวณ similarity matrix ขนาด [batch_size, batch_size]
22         similarity_matrix = torch.matmul(features, features.t()) /
self.temperature
23
24         # สร้าง mask สำหรับตัวอย่างเดียวกัน (ไม่เอา self-similarity)
25         self_mask = torch.eye(batch_size, dtype=torch.bool,
device=device)
26
27         loss = 0.0
28         count = 0
29
30         for i in range(batch_size):
31             # เลือก indices ของ positive samples: มี label เดียวกับ i แต่
ไม่ใช่ i เอง
32             pos_mask = (labels == labels[i]) & (~self_mask[i])
33             if pos_mask.sum() == 0:
34                 continue # หากไม่มี positive pair สำหรับ sample นี้
35             # ค่าคล้ายคลึงของ sample i กับตัวอย่างอื่นๆ (ยกเว้นตัวเอง)
36             logits = similarity_matrix[i][~self_mask[i]]
37             # ค่าคล้ายคลึงของคู่ positive อยู่ใน pos_mask
38             pos_logits = similarity_matrix[i][pos_mask]
39             # คำนวณ denominator สำหรับ softmax
40             denom = torch.sum(torch.exp(logits))
41             # Loss สำหรับ sample i คือค่าเฉลี่ยของ -log(probability) สำหรับ
แต่ละ positive
42             loss_i = - torch.log(torch.exp(pos_logits) / denom)
43             loss += torch.mean(loss_i)
44             count += 1
45
46         return loss / count if count > 0 else loss

```

```
1 features = torch.randn(6, 8)
2 labels = torch.tensor([0, 0, 1, 1, 2, 2])
3
4 criterion = SupNTXentLoss(temperature=0.5)
5 loss = criterion(features, labels)
6 print("Supervised NT-Xent Loss:", loss.item())
```

สรุป

Loss functions เหล่านี้มีบทบาทสำคัญในการผลักดันให้โมเดลเรียนรู้ representation ที่สามารถแยกแยะความแตกต่างระหว่างคลาสได้อย่างชัดเจน

- Contrastive Loss และ Max Margin Contrastive Loss ใช้แนวคิดการดึง positive pairs เข้ามาใกล้กันและผลัก negative pairs ให้ห่างออกไป
- Triplet Loss ใช้การเปรียบเทียบระหว่างชุด triplet (anchor, positive, negative) เพื่อให้เกิดความแตกต่างที่ชัดเจน
- Multi-class N-pair Loss ขยายแนวคิดของ Triplet Loss โดยใช้ข้อมูลเชิงลบหลายตัวในคราวเดียว
- Supervised NT-Xent Loss ต่อยอดจาก NT-Xent loss แบบ self-supervised ด้วยการนำข้อมูล label เข้ามาช่วยให้การจับคู่ positive และ negative มีความแม่นยำยิ่งขึ้น

การเลือกใช้ loss function ที่เหมาะสมขึ้นอยู่กับลักษณะของชุดข้อมูลและเป้าหมายของงาน เช่น งาน classification, clustering หรือ retrieval ซึ่งแต่ละแบบมีข้อดีและข้อจำกัดแตกต่างกันไป